
GENERATIVE ZERO-SHOT LEARNING FOR DECODING VISUAL STIMULI EEG DATA

A PREPRINT

Jacob W. McKerrell
qbvc46@durham.ac.uk

December 13, 2025

ABSTRACT

Human identification of visual stimuli fundamentally makes use of the distribution of neuron activity. The brain’s challenge is to take these complex electrical signals, represented in the neural space $\mathcal{X}$, and efficiently identify the underlying meaning or concept it represents based on its descriptive features. If we have an understanding of this meaning, represented in the semantic space $\mathcal{A}$, it allows us to identify and label the object ($\mathcal{Y}$), regardless of whether we have seen an object like it before. The ultimate goal of this project is to implement a generative Zero Shot Learning (ZSL) model, which will replicate this process to find a Neural-to-category decoding ($\mathcal{X} \rightarrow \mathcal{Y}$). Our model was trained exclusively on 40 seen classes, then evaluated on a test set where the classifier must choose a category, from the set of $(Y_{seen} \cup Y_{unseen})$. This set is the augmentation of our 40 trained categories with 20 unseen categories. Inspired by human imagination, we implemented a generator to learn the mapping function ($g : \mathcal{A} \rightarrow \mathcal{X}$). This enables the synthesis of neural patterns for concepts it has never observed ($\hat{X}_{unseen}$) enabling the final classification ($c : \mathcal{X} \rightarrow \mathcal{Y}$) to be effective on these concepts. We compare two implementations of our generator, which both calculate a prototype from the average semantic description of a category and use a Gaussian noise distribution to add a cloud of points around our calculated average brain map, which accounts for variation in different objects within the same category. Our baseline generator uses a linear regression, and produces an unseen accuracy of $\sim 3.38\%$. To combat the overfitting, and low accuracy that this model was subject to, we implemented a custom, non-linear, Kernel Ridge Regression (KRR) generator. Accuracy of unseen classes showed a clear improvement to $\sim 6.5\%$. These results provide evidence that a generative ZSL framework can learn a meaningful—though limited—cross-modal mapping between semantic and neural spaces, enabling improved generalisation to unseen categories.

1 Introduction

Classification is a fundamental Machine Learning (ML) task, that involves assigning input data samples to predefined discrete classes. Supervised learning is a traditional Machine Learning technique which can be used to classify a large sample of labelled training data. The range of classifications it can give as output, is limited to the classes it was exposed to in the training data. For projects such as our universal brain reader, it is unfeasible to collect labelled training data for every possible category that could exist; we must implement a more advanced ML paradigm which can generalise to an ever-growing set of classes [8]. Yang Long’s implementation of a generation function, to mimic human imagination, allows models to classify inputs into previously unseen classes (Y_{unseen}), based on exposure solely to their semantic descriptions. In his later paper [9], the idea is extended, by enabling the synthesis of information for entirely unseen categories

1.1 Related Work

An advanced alternative to traditional Machine Learning techniques, is deep learning. Driven by multilayered neural networks and inspired by the human brain, each individual “neuron” performs its own mathematical operation. Through fine-tuning of the weights and biases of neurons, the network is optimized to give the desired outputs [2]. This has the advantage of high adaptability, which is critical when modelling high-dimensional and noisy data. Whilst deep learning models have not been directly implemented in this paper, our data has been preprocessed using this technology. Most notably, CLIP utilises deep learning to obtain high-quality semantic description data. Machine Learning technology extends far beyond classification. Pu et al. [13] combine reinforcement learning and deep learning to mimic human evolution to form a perpetually adaptive set of rules for a model to follow. Whilst not directly relevant to our paper, these ideas show the shift towards paradigms being able to generalise beyond simple static training sets.

2 DATA and PARADIGM

2.1 Exploration of the Dataset

Things-EEG-Text is a trimodal dataset, covering visual features, textual features and brain features, obtained through EEG recordings from 10 individuals exposed to the visual stimulus of the images [3]. Things-EEG-Text does not contain raw data, but it has already been processed: The textual features have been processed using CLIP (Contrastive Language-Image Pre-training), which is a neural network made by openai, designed to efficiently learn visual concepts from natural language supervision [14]. The Visual features have been pre-processed using CORnet-s [6]. The Brain features data were recorded using a 64-channel EASYCAP, which has been down-sampled to 17 channels, once again for computational simplicity. Additionally, these 17 channels have been specially selected so that it is the channels representing the mainly visually sensitive components of the brain which have been kept [3]. Each of these channels represents a different position of the brain. We will extract the data found in the time interval 70ms – 400ms post stimulus, composed of 33 discrete samples. Overall there are 561 brain features and 512 text features available for use.

2.2 Identifying imbalances and correlations

2.2.1 Analysing the spread and topological positioning of the classes

Our data has a uniform category distribution (each individual subject was exposed to 10 images of each concept / category [3].) We will maintain this even split in our training data. Uneven split of classes would cause bias towards more frequent classes, so it is essential to remove this bias, for any Machine Learning implementation. We also need to verify whether the Y_{unseen} in THINGS-EEG-text are sufficiently distinct from the existing Y_{seen} . We decided to compare the data distribution between Y_{seen} and Y_{unseen} using a t-SNE plot. T-distributed Stochastic Neighbour Embedding (t-SNE) "converts similarities between data points to joint probabilities and tries to minimize the Kullback-Leibler divergence between the joint probabilities of the low-dimensional embedding and the high-dimensional data" [11]. Essentially, we use the t-SNE plot to represent the high dimensional semantic features vector of an averaged instance of each category on a 2D grid. It shows the topology of the data, so datapoints which share a similar feature composition are clustered together on this grid. We implemented the t-SNE algorithm using the scikit-learn Python module [11]. We can see from the plot, that members of Y_{unseen} are found dispersed between the members of Y_{seen} suggesting there is a wide and diverse spread of data classes, ideal for ZSL. If the members of Y_{unseen} were placed directly on top of members of Y_{seen} , then there is evidence to suggest the classes are not semantically distinct, and in this situation ZSL will struggle, due to its bias towards members of Y_{seen} . This bias means that while the trained model may achieve high performance on the data of Y_{seen} , the performance will degrade when using Y_{unseen} data [8]. The other alternative is a situation where Y_{unseen} have t-SNE co-ordinates which form a cluster drastically separated from the Y_{seen} classes. If this were the case, the implication would be that there is insufficient information to deduce what the new Y_{unseen} would be, as it is not similar to any existing Y_{seen} which would have been used in the training process. Thankfully, this is not the case, and so a ZSL model becomes a feasible option: We have evidence to suggest identifying a member of Y_{unseen} is possible.

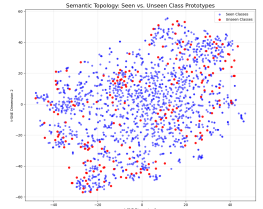


Figure 1: t-SNE plot

2.2.2 Analysing the correlation between the Semantic, and Neural spaces

We need to see if there is a mapping between the Neural space ($\mathcal{X} = \{x_1, x_2, \dots, x_n\}, x_i \in \mathbb{R}^{561}$) and the semantic space ($\mathcal{A} = \{a_1, a_2, \dots, a_n\}, a_i \in \mathbb{R}^{512}$). The simplest model to initially test, would be a linear relationship, where classes that are neighbours in the semantic space, are also neighbours in the neural space. When we plot the distance of each category from the category indexed 0, we find a very low PCC value of 0.2496 (Figure 2). This low correlation is likely due to the complexity of the two spaces, as well as the noise present, especially in $\mathcal{X}$. The high dimensionality of our data structures can also cause the "Curse of dimensionality" [12]. All these factors suggest a non-linear relationship between $\mathcal{A}$ and $\mathcal{X}$, making a linear-mapping solution likely inadequate. In order for our ZSL paradigm to be a feasible option, we need it to be able to synthesize neural features. We want to find a mapping function from the semantic space to the neural space ($f : \mathcal{A} \rightarrow \mathcal{X}$) [7] to allow the semantic data to be sufficient information to accurately "hallucinate" brain features [8]. Figure 3 shows the self-similarity matrices for both $\mathcal{X}$ and $\mathcal{A}$. Whilst the graph demonstrates rather noisy data, especially in $\mathcal{X}$, it is clear that there are features within each space that uniquely define each class. We have calculated a relatively low average cosine similarity score $S = \frac{1}{n(n-1)} \sum_{i=1}^n \sum_{j=1, j \neq i}^n |S_{ij}|$ where n is the number of classes and S_{ij} is the cosine similarity between classes i and j : $S_{brain\ features} = 0.139857$, $S_{text\ features} = 0.090472$. These low values display the class distinctiveness in both $\mathcal{X}$ and $\mathcal{A}$ spaces. This gives us a strong foundation for potential generalisation, furthering our confidence in our generative ZSL paradigm. When an unseen semantic vector $\mathcal{A}_{unseen}$ is passed as input, the learned mapping (which we now know should be possible) will be able to produce an accurate synthesized brain pattern. It is important to note, that there is also only weak correlation between EEG data

from different individuals [4]. Our decision to adopt a single-subject paradigm is justified by the requirement of controlling data noise to improve model performance. By restricting the scope to a single, subject, we strategically minimize noise we would get from inter-subject variability, to the more manageable level of intra-subject variability.

2.3 Paradigm design and real world value

We want a paradigm, that is able to take a new EEG brain scan as an input, and from this identify the visual stimulus that the brain has been looking at and output the category of object that it has seen, much like what a real human brain can do. A traditional Machine learning paradigm, such as supervised learning, is not suitable for this problem, due to its limitation of having to pre-define and train with all of its classes. [10] Completing the process of measuring brain signals for every possible visual stimulus class, is not only expensive, but essentially impossible. You cannot strap an EEG cap on a person and show them every possible object in the universe. We need a scalable way to build a universal brain reader, which requires a system capable of “hallucinating” brain signals from just the textual description. ZSL allows the problem to become feasible again as it allows new objects to be recognised, regardless of whether physical scans have taken place. All that it needed to identify new objects is their semantic description[8]. This makes the ZSL paradigm a perfect fit for our model. There is an issue however, that arises from a standard ZSL implementation, where there is a bias towards members of Y_{seen} , as outlined in section 2.2.2. This bias is caused, because the data is trained exclusively on members of Y_{seen} , causing it to systematically classify new inputs into already known classes. Therefore, I aim to implement a generative zero shot learning approach, similar to the UVDS method used by Yang Long [8]. The idea is to “imagine” features of the brain patterns that do not actually exist (X_{unseen}). This reduces the natural bias towards the seen data, because the model is trained with both $Y_{seen} \cap Y_{unseen}$. In order to calculate X_{unseen} , we will use existing brain data, to predict what a brain scan would look like for a member of Y_{unseen} . Since I am using ZSL, I require 2 separate splitting strategies. First is the Disjoint category split. We define our seen set $\mathcal{S} = \{(x, y) | y \in Y_{seen}\}$ and the unseen set $\mathcal{U} = \{(x, y) | y \in Y_{unseen}\}$ Where $Y_{seen} \cap Y_{unseen} = \emptyset$. This disjoint property is essential for a ZSL implementation. This split comes directly from the dataset: 1654 members of Y_{seen} and 200 members of Y_{unseen} . Secondly we have the internal validation split. We will split this in a 70/30 ratio, so that the majority of $\mathcal{S}$ is used for training purposes, whilst the remainder will be used to test that the algorithm responsible for “hallucinating” new brain data features is reliable and that the model generalises effectively, before exposing it to any actual member of Y_{unseen} . It is important that we stratify based on Y_{seen} so that there are 7 samples used in each seen category for training and 3 for testing. As part of my implementation, I will also use PCA (principal component analysis) on our semantic to reduce the dimensionality of a data set which currently consists of a large number of interrelated variables [5]. We will extract the most relevant features from the semantic data, so that we maintain 95% of the total variance. This technique not only reduces computational requirements, but also manages to remove some of the noise that is present in Figure 3. We will use LDA to reduce the brain features. LDA aims to maximize the separation between multiple classes. It keeps the dimensions that make the classes distinct from one another.[1]. LDA limited to a maximum of $C - 1$ discriminant axes, so if we had 40 classes, the maximum number of dimensions that we could reduce our brain data to, would be 39 [11]. We will use 40 seen classes, and 20 unseen classes, for the implementation of our ZSL algorithm. These match the classes shown in Figure 3. Using this reduced set, allows for a more accurate model, since our dataset limits us to 10 samples for each category (which we have split into 7 for training and 3 for testing), we don’t have enough information to successfully differentiate 1854 classes.

3 MODEL and DEVELOPMENT

3.1 Implementation of a baseline model

Our baseline model implements our generator function using scikit-learn’s linear regression [11]. This model assumes $\hat{X} = AW$ maps our PCA and LDA processed data $\mathbb{R}^{102} \rightarrow \mathbb{R}^{39}$, and we want to minimise the Mean Squared Error (MSE): $\min_W ||X - AW||^2$. As we discovered in Figure 2, there is a very weak linear correlation between classes in the semantic and neural spaces, so this model was designed to be intentionally conservative with its results. Evaluation of the linear generator shows a low reconstruction correlation, and modest cosine similarities (0.15). This validated our prediction that a linear mapping model would be insufficient for successfully mapping the cross-modal transformation. The weak correlation we have between $\mathcal{A}$ and $\mathcal{X}$ will limit the accuracy of the synthesized brain features produced. In order to maintain the intra-category variability in the synthesis of brain features, we found the semantic category prototype for each class. We passed this single point into the generator function, which gave us a single average predicted neural point for each category (μ_c). We want to maintain the variance (σ_c^2) associated with the training brain data (which we calculated to be 0.9526), so we add Gaussian noise to create a cloud of neural samples based

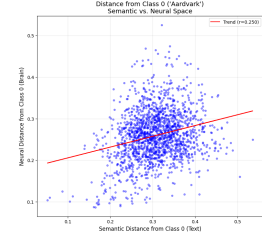


Figure 2: Pearson's correlation plot

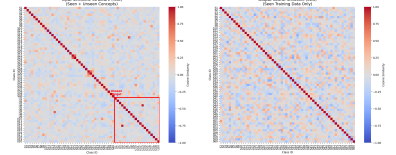


Figure 3: Heatmaps showing inter-category relationships using RSA for semantic and neural data. for 60 classes

on each of our prototypes $\hat{x} \sim N(\mu_c, \sigma_c^2)$. For the classification stage, we used scikit-learn’s L2 regularized softmax logistic regression function [11], which was trained on our training split of X_{seen} augmented with the complete set of synthesized unseen features $\hat{X}_{unseen}$.

3.2 Implementation of custom model from scratch

For our custom model, we are updating our linear regression to a kernel ridge regression (KRR) method. Kernel Ridge Regression uses a kernel function, which computes the inner product between patterns, after they have been mapped to a high-dimensional space [15]. The general form of our non-linear generator between defined as: $f^{ML}(a) = \sum_{j=1}^{N_T} \alpha_j k(a, a_j)$, where a is the semantic input, α is the matrix of learnt weights (calculated by the minimisation function), k is the kernel and λ is the strength of regularization (Larger λ implies more noise around the training points [15]). We will implement this mapping function by making use of the matrix form $K^* \times \alpha$ where K^* is our testing data. We will be using the Gaussian kernel function: $k(a, a') = \exp(-\frac{(a-a')^2}{2\sigma^2})$ [15]. Where σ is

the length scale of the Gaussian. It uses the minimisation function: $\mathcal{C}(\alpha) = \sum_{j=1}^{N_T} (f^{ML}(a_j) - x_j)^2 + \lambda \alpha^T K \alpha$ [15]. The first term is the sum of the squared differences between the model’s prediction and the training points, and the true training points. The second term acts as a regularisation and imposes a penalty on the complexity of the learned function this is the "ridge" part of KRR. Given the noisy nature of our EEG data, this regularisation is critical; It prevents the model from overfitting to the specific training data, instead, it is forced to calculate a generalised mapping. To calculate our vector of weights $\alpha = (\alpha_1, \dots, \alpha_T)^T$ we use $(K + \lambda I)^{-1} f$. KRR doesn’t converge over time, but instead uses a closed-form solution, which is calculated using a matrix inversion ($O(N^3)$). With our relatively small, stripped-back dataset size, this remains a feasible solution.

We found that our custom model result, using a non-linear generator, showed only mild improvement over the linear model. With a cosine similarity score of 0.1742, the change is limited compared to the baseline. This suggests there is a bottleneck within the structure of our data. A sample size of 10 (of which we only used 7 for training) is insufficient for the complex and noisy structure of the brain data, meaning we were unable to produce an accurate mapping between $\mathcal{A} \rightarrow \mathcal{N}$.

3.3 Methodological improvements and hyperparameter configuration

We needed to tune our hyperparameters (λ and σ) in order to get the best-fitting model. We used GridSearchCV [11] to train the model on a variety of parameter values. λ and σ are the external hyper-parameters, whilst values of α_j are calculated within the fitting process. We scored our tuning based on cosine similarity, with the aim of maximising the similarity of the synthesised vectors, to the actual vectors of the brain signals they represent. We chose to use cosine similarity as our measure, rather than Euclidean distance, since the orientation and direction of vectors in high-dimensional space carry more information than the magnitude, so vectors with similar features will have comparable orientations in this space. Our hyperparameter calculated $\lambda = 0.01$ and $\sigma = 5$ to be our optimal values finding the best balance between regularisation strength with kernel sensitivity.

Since one of our largest limitations, is the quantity of data we have been given (X_{seen} and A_{seen}) one methodological improvement, which we could implement in a future project, would be the addition of data augmentation from the semantic and neural data samples in Y_{seen} . These new sample vectors will be based on the mean and standard deviation of the category samples we do have, but with an added Gaussian noise (much like the process we used for synthesising the unseen data). This would give a clearer cluster for each class, potentially improving the accuracy of the classification. If we were to have more data samples (preferably real), it would make a deep learning based neural network a more feasible option. A system like this would be able to find more sensitive mappings between the Neural and Semantic spaces, improving upon our linear and KRR implementations.

4 RESULT ANALYSIS

4.1 Evaluation of performance

Our complete ZSL baseline model accuracy for X_{seen} inputs was $\sim 8.33\%$ and $\sim 3.38\%$ for X_{unseen} . While these values may seem initially negligible or irrelevant, it is important to consider true random baseline accuracy is $\frac{1}{Y_{seen} + Y_{unseen}} = 1.6\%$. The calculated accuracy for both models, is significantly larger than this, which suggests that even the baseline implementation is finding some sort of relationship, which the custom model later builds on. However, there is a large discrepancy in the accuracy for X_{unseen} and X_{seen} within the baseline implementation. This suggests that our linear synthesis function is strongly overfitting to the seen classes, making unseen generalisation very weak.[8]. Our custom model successfully traded memorization for generalization, increasing X_{unseen} accuracy to $\sim 6.5\%$, but reducing X_{seen} accuracy to $\sim 4.17\%$. These observations support our earlier hypothesis that the baseline model was overfitting to the classes it had seen before and was ineffective at generalizing to unseen classes. Further analysis of the F1-scores (which measures the balance between the model’s precision and recall) confirms this issue has been reduced; X_{seen} has decreased, whilst X_{unseen} has increased.

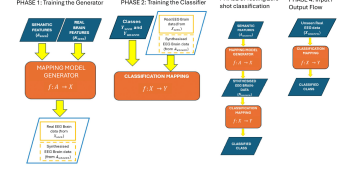
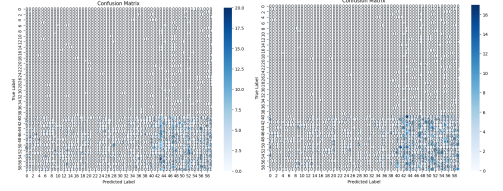


Figure 4: Paradigm Flow Diagram

We want to test whether the baseline model has higher accuracy at classifying X_{unseen} inputs compared to random chance. We performed a hypothesis test with $H_0 : p = 0.01666$ and $H_1 : p > 0.01666$ where p is our probability of correctly identifying a X_{unseen} input. Comparing our baseline model’s calculated accuracy of $\sim 3.38\%$, with 60 classes and 1600 unseen samples, we found there was sufficient evidence to reject H_0 at the 1% significance level, which gave sufficient evidence to suggest that our baseline model is not random.

4.2 Visualisation of model

Analysis of the confusion matrices in Figure 5 reveals some improvement in precision. Both models have a weak, but present, diagonal across the whole matrix. However, in the baseline model 5(a), there is a larger systematic misclassification between X_{seen} inputs and Y_{seen} outputs. The custom model 5(b) is less likely to predict a seen class, but when it does predict a seen class it is more likely to be the correct one. (Baseline: 30% Custom: $\sim 38\%$), a clear depiction of the seen class bias. When the custom model encounters uncertainty, it no longer defaults the prediction to a Y_{seen} category rather it will more commonly predict a member of Y_{unseen} in this situation, showing that generalisation of the model has improved. The density of true positives in the custom model concentrated in the bottom right block (Actual Y_{unseen} vs. Predicted Y_{unseen}) shows the effectiveness of unseen class classification. The model can reliably determine whether an X_{unseen} is actually unseen. Our confusion matrices also highlight the limitations of our models. There are classes for which similar $\hat{X}_{unseen}$ have been produced. Notably, class 40 inputs (Aircraft carrier) were misclassified to class 43 (balance beam) a total of 10 times in the baseline and 17 times in the custom. These two classes have similar $\mathcal{A}_{unseen}$ so classification is often confused. Consider the long, flat and uniform appearance and dominance of straight edges, which define both of these objects; the models’ confusion seem plausible. Finally, the X_{seen} results have been heavily influenced by severe test set imbalance. The Y_{seen} test set (containing 3 samples) is substantially smaller than the 80 samples per class present in X_{unseen} test data. This lack of data means we do not get the entire picture of how the model behaves for Y_{seen} . Implementation of the X_{seen} synthesis outlined in section 3.3 may mitigate this issue.



(a) Baseline (linear) (b) Custom (KRR)

Figure 5: Confusion Matrices

Table 1: Model Performance Summary (Seen and Unseen)

Metric	Seen	Unseen	Overall
Baseline			
Precision	0.1708	0.0480	0.0545
Recall	0.0833	0.0338	0.0471
F1-Score	0.1083	0.0355	0.0427
Accuracy	0.0833	0.0338	0.0471
Custom			
Precision	0.1000	0.0786	0.0690
Recall	0.0417	0.0650	0.0634
F1-Score	0.0575	0.0691	0.0635
Accuracy	0.0417	0.0650	0.0634

References

- [1] D. Blei, A. Ng, and M. Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.
- [2] H. Duan, Y. Long, S. Wang, H. Zhang, C. G. Willcocks, and L. Shao. Dynamic unary convolution in transformers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11):12747–12759, 2023.
- [3] A. T. Gifford, K. Dwivedi, G. Roig, and R. M. Cichy. A large and rich eeg dataset for modeling human visual object recognition. *NeuroImage*, 264:119754, 2022.
- [4] G. Huang, Z. Zhao, S. Zhang, Z. Hu, J. Fan, M. Fu, J. Chen, Y. Xiao, J. Wang, and D. Guo. Discrepancy between inter- and intra-subject variability in eeg-based motor imagery brain-computer interface: Evidence from multiple perspectives. *Frontiers in Neuroscience*, 17, 02 2023.
- [5] I. T. Jolliffe. *Introduction*, pages 1–9. Springer New York, New York, NY, 2002.
- [6] J. Kubilius, M. Schrimpf, H. Hong, N. J. Majaj, R. Rajalingham, E. B. Issa, K. Kar, P. Bashivan, J. Prescott-Roy, K. Schmidt, A. Nayebe, D. Bear, D. L. K. Yamins, and J. J. DiCarlo. Brain-Like Object Recognition with High-Performing Shallow Recurrent ANNs. In *Neural Information Processing Systems (NeurIPS)*, pages 12785–12796, 2019.
- [7] Y. Long, L. Liu, L. Shao, F. Shen, G. Ding, and J. Han. From zero-shot learning to conventional supervised classification: Unseen visual data synthesis, 2017.
- [8] Y. Long, L. Liu, F. Shen, L. Shao, and X. Li. Zero-shot learning using synthesised unseen visual data with diffusion regularisation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PP:1–1, 10 2017.
- [9] Y. Long, L. Liu, Y. Shen, and L. Shao. Towards affordable semantic searching: Zero-shot retrieval via dominant attributes. 03 2018.
- [10] V. Nasteski. An overview of the supervised machine learning methods. *HORIZONS.B*, 4:51–62, 12 2017.
- [11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011.
- [12] D. Peng, Z. Gui, and H. Wu. Interpreting the curse of dimensionality from distance concentration and manifold effect, 2025.
- [13] J. Pu, H. Duan, J. Zhao, and Y. Long. Rules for expectation: Learning to generate rules via social environment modeling. *IEEE Transactions on Circuits and Systems for Video Technology*, 34(8):6874–6887, 2024.
- [14] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. *CoRR*, abs/2103.00020, 2021.
- [15] K. Vu, J. Snyder, L. Li, M. Rupp, B. F. Chen, T. Khelif, K.-R. Müller, and K. Burke. Understanding kernel ridge regression: Common behaviors from simple functions to density functionals, 2015.